Practice · Whole programs 3

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What the table program does

Read the whole program before you answer.

```
#include <stdio.h>
int main(void) {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n * i);
    }
    return 0;
}
```

What does it do?

- It multiplies two numbers that you type.
- **Right answer:** It asks for one number and prints its multiplication table, from times 1 to times 10.
- It prints the numbers from 1 to 10.
- It asks for ten numbers and multiplies them.

Feedback: Ask, read, repeat. `scanf` reads `n` once. The `for` loop then runs its block 10 times, and each time the `printf` prints one line of the table. One `printf` in the code, ten lines on the screen. This is class 6, slide 58.

(mc) L1 · What changes and what stays

Same program:

```
#include <stdio.h>
int main(void) {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n * i);
    }
    return 0;
}
```

While the loop runs, which variable changes and which stays the same?

- `n` changes, `i` stays the same.
- **Right answer:** `i` changes from 1 to 10, `n` stays the same.
- Both change on every repeat.
- Neither changes.

Feedback: `i` is the loop counter: the update `i++` makes it 1, 2, 3, ... 10. Nothing inside the loop touches `n`, so it keeps the value you typed. Each line uses the current `i`, which is why every line is different.

(mc) L1 · The shape of a loop program

Most programs this week have the same steps in the same order. Which order?

- Repeat, ask, read, print the result.
- **Right answer:** Ask, read, repeat a block, print the result.
- Ask, print the result, read, repeat.
- Read, ask, print the result, repeat.

Feedback: Look at the sum program: ask how many values, read `n`, repeat "read a value and add it" `n` times, then print the total after the loop. Weeks 1 and 2 were ask, read, calculate or decide, print. Now the middle step happens many times.

(mc) L1 · What the loop controls in the counting program

```
#include <stdio.h>
int main(void) {
    int count = 0;
    int n, value;
    printf("How many values? ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        if (value > 0) {
            count++;
        }
    }
    printf("Positive values: %d\n", count);
    return 0;
}
```

What does the `for` loop control?

- How big the values may be.
- **Right answer:** How many values are read: the block runs `n` times, so `scanf` reads `n` values.
- Whether a value is positive.
- How many times the result is printed.

Feedback: `i` goes from 0 to `n - 1`, so the block runs exactly `n` times. Each run reads one value. The `if` inside decides whether that value is counted. The result is printed once, after the loop. Class 6, slide 26.

(mc) L1 · Which line adds to the total

```
#include <stdio.h>
int main(void) {
    int n, value;
    int sum = 0;
    printf("How many values? ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &value);
        sum += value;
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

Which line makes the total grow?

- Line 4, `int sum = 0;`
- Line 8, `scanf("%d", &value);`
- **Right answer:** Line 9, `sum += value;`
- Line 11, `printf("Sum = %d\n", sum);`

Feedback: `sum += value;` adds the value just read to the running total. Line 4 only makes the box and puts 0 in it. Line 8 reads. Line 11 shows the result once, at the end. This is task 3 on class 6, slide 85.

(mc) L1 · Why while in the age program

```
#include <stdio.h>
int main(void) {
    int age;
    printf("Enter age: ");
    scanf("%d", &age);
    while (age < 0 || age > 120) {
        printf("Invalid. Try again: ");
        scanf("%d", &age);
    }
    printf("Accepted\n");
    return 0;
}
```

Why does this program use `while` and not a counted `for`?

- `for` cannot contain `scanf`.
- `while` is faster.
- **Right answer:** Nobody knows how many wrong ages the user will type, so the loop must go on until the input is good.
- `for` cannot use `||`.

Feedback: A `for` fits when you can count the repeats before the loop starts. Here the number of repeats depends on the user: maybe 0 wrong tries, maybe 7. `while` says "as long as the age is bad, ask again". Class 6, slide 36.

(mc) L1 · When the password loop stops

```
#include <stdio.h>
int main(void) {
    int code;
    printf("Password: ");
    scanf("%d", &code);
    while (code != 1234) {
        printf("Wrong. Try again: ");
        scanf("%d", &code);
```

```
    }
    printf("Welcome\n");
    return 0;
}
```

When does the loop stop?

- After three wrong tries.
- When `code` is not 1234.
- **Right answer:** When `code` is 1234, because then `code != 1234` is false.
- It never stops.

Feedback: `!=` means "is not equal to". The loop goes on **while** the code is wrong. As soon as you type 1234, the condition is false, the loop ends, and `Welcome` is printed. This is task 5 on class 6, slide 85, with a number as the password.

(mc) L1 · What fact *= i does

```
#include <stdio.h>
int main(void) {
    int n;
    int fact = 1;
    printf("n: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    printf("%d! = %d\n", n, fact);
    return 0;
}
```

What does `fact *= i;` do on each repeat?

- It sets `fact` to `i`.
- It adds `i` to `fact`.
- **Right answer:** It multiplies `fact` by the current `i` and keeps the result in `fact`.
- It multiplies `i` by 2.

Feedback: `fact *= i;` is `fact = fact * i;`. For `n` = 5, `fact` goes 1, 1, 2, 6, 24, 120. It is the accumulator pattern with `*` instead of `+`, so it starts at 1 and not at 0. Class 6, slide 63.

(mc) L1 · The step of 2

```
#include <stdio.h>
int main(void) {
    int n;
    printf("n: ");
    scanf("%d", &n);
    for (int i = 2; i <= n; i += 2) {
        printf("%d ", i);
    }
    printf("\n");
    return 0;
```

```
}
```

What does the update `i += 2` do?

- It prints `i` twice.
- It multiplies `i` by 2.
- **Right answer:** It adds 2 to `i` after every repeat, so `i` is 2, 4, 6, ...
- It stops the loop after 2 repeats.

Feedback: The update does not have to be `i++`. Starting at 2 and adding 2 each time visits only the even numbers. This is task 2 on class 6, slide 85.

(mc) L2 · The last line of the table for 7

Use the table program (class 6, slide 58). You type `7`. What is the **last** line printed?

- `7 x 7 = 49`
- `7 x 9 = 63`
- **Right answer:** `7 x 10 = 70`
- `7 x 11 = 77`

Feedback: The condition is `i <= 10`, so the last repeat has `i` equal to 10 and prints `7 x 10 = 70`. Then `i` becomes 11, `11 <= 10` is false, and the loop ends.

(mc) L2 · The fifth line of the table for 3

Use the table program again. You type `3`. What is the **fifth** line printed?

- `3 x 3 = 9`
- `5 x 3 = 15`
- **Right answer:** `3 x 5 = 15`
- `3 x 5 = 8`

Feedback: On the fifth repeat `i` is 5. The `printf` has three holes filled with `n`, `i` and `n * i`, in that order: `3 x 5 = 15`.

(mc) L2 · Counting positives

Use the counting program (class 6, slide 26). You type `4` for "How many values?" and then the values `5 -2 0 9`. What is printed at the end?

- `Positive values: 4`
- `Positive values: 3`
- **Right answer:** `Positive values: 2`
- `Positive values: 1`

Feedback: Only 5 and 9 pass `value > 0`. -2 is negative. 0 is not greater than 0, so it is not counted either. The border case is always worth testing.

(mc) L2 · The sum of three values

Use the sum program. You type `3` for "How many values?" and then `10 20 5`. What is printed?

- `Sum = 3`

- `Sum = 30`
- **Right answer:** `Sum = 35`
- `Sum = 38`

Feedback: `sum` starts at 0 and grows: 10, 30, 35. The 3 you typed first is `n`, the number of values. It is not added to the sum.

(mc) L2 · Even numbers up to 9

Use the even-numbers program. You type `9`. What is printed after `n:` ?

- `2 4 6 8 10`
- **Right answer:** `2 4 6 8`
- `1 3 5 7 9`
- `2 4 6 8 9`

Feedback: `i` is 2, 4, 6, 8, and each is `<= 9`. Then `i` becomes 10, `10 <= 9` is false, and the loop ends. With `10` typed you would also get 10. With `1` typed the loop runs zero times and only an empty line is printed.

(mc) L2 · The tracing exercise

```
int x = 2;
int y = 0;
while (x < 8) {
    y += x;
    x += 2;
}
printf("x=%d y=%d", x, y);
```

What exactly is printed?

- `x=6 y=12`
- `x=8 y=20`
- **Right answer:** `x=8 y=12`
- `x=8 y=6`

Feedback: Make a table. Start: x=2, y=0. After repeat 1: y=2, x=4. After repeat 2: y=6, x=6. After repeat 3: y=12, x=8. Now `8 < 8` is false, so the loop ends. The block ran 3 times, and `x += 2;` is the line that moves the loop toward stopping. Class 6, slide 78.

(mc) L2 · The tracing exercise with new numbers

```
int x = 1;
int y = 0;
while (x < 10) {
    y += x;
    x += 3;
}
printf("x=%d y=%d", x, y);
```

What exactly is printed?

- `x=7 y=12`

- **Right answer:** `x=10 y=12`
- `x=10 y=22`
- `x=9 y=12`

Feedback: x=1, y=0. After repeat 1: y=1, x=4. After repeat 2: y=5, x=7. After repeat 3: y=12, x=10. Then `10 < 10` is false. Do not guess: write down the values after every repeat.

(mc) L2 · Find the bug: the factorial that is always 0

```
#include <stdio.h>
int main(void) {
    int n;
    int fact = 0;
    printf("n: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    printf("%d! = %d\n", n, fact);
    return 0;
}
```

It compiles with no warning. You type `5` and see `5! = 0`. What is the bug?

- The loop should start at `i = 0`.
- **Right answer:** `fact` starts at 0, and 0 times anything is 0. It must start at 1.
- `fact *= i;` should be `fact += i;`.
- `%d` should be `%lf`.

Feedback: A product must start at 1, because multiplying by 1 changes nothing. Starting at 0, every `fact *= i;` gives 0 again. With `int fact = 1;` the program prints `5! = 120`.

(mc) L2 · Find the bug: the sum that forgets

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
    sum = 0;
    sum += i;
}
printf("Sum = %d\n", sum);
```

It should print `Sum = 15`, but it prints `Sum = 5`. What is the bug?

- The loop should run to 15.
- `sum += i;` should be `sum++;`.
- **Right answer:** `sum = 0;` inside the loop throws the total away on every repeat, so only the last `i` is left.
- `printf` should be inside the loop.

Feedback: The total must keep its value across the repeats. Set it to 0 **once, before** the loop. Here it is reset five times, and after the last reset only 5 is added. Class 6, slide 71 calls this "resetting inside the loop".

(mc) L2 · Find the bug: = in the condition

```
#include <stdio.h>
int main(void) {
    int i = 1;
    while (i = 3) {
        printf("%d\n", i);
        i++;
    }
    return 0;
}
```

`gcc -Wall` warns on line 4: `suggest parentheses around assignment used as truth value`. What happens when you run it?

- It prints `1`, `2`, `3` and stops.
- It prints `3` once and stops.
- **Right answer:** It prints `3` again and again and never stops.
- It prints nothing.

Feedback: `i = 3` is not a question. It puts 3 into `i`, and 3 is not zero, so C treats it as true, every time. The `i++` makes `i` 4, and the next check makes it 3 again. Stop it with Ctrl+C and write `i <= 3`. You met this warning in week 2: it almost always means a missing `=` or a wrong operator.

(fib) L3 · Complete the table: the condition

Complete the condition so the table goes from times 1 to times 10. Type it with single spaces, exactly like `i <= 5`.

```
#include <stdio.h>
int main(void) {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (int i = 1; [i <= 10|i<=10|i < 11|i<11]; i++) {
        printf("%d x %d = %d\n", n, i, n * i);
    }
    return 0;
}
```

Feedback: The block must still run when `i` is 10, so `i <= 10`. With `i < 10` the table would stop at times 9.

(fib) L3 · Complete the table: the product

Complete the third value so each line shows the right result. Type it with single spaces, exactly like `a * b`.

```
for (int i = 1; i <= 10; i++) {
    printf("%d x %d = %d\n", n, i, [n * i|n*i|i * n|i*n]);
}
```

Feedback: Three holes, three values: `n`, then `i`, then the product `n * i`. For `n` = 7 and `i` = 2 the line is `7 x 2 = 14`.

(fib) L3 · Complete the counter: the starting value

Complete the starting value of the counter. Type only the number.

```
int count = [0];
int n, value;
printf("How many values? ");
scanf("%d", &n);
for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    if (value > 0) {
        count++;
    }
}
printf("Positive values: %d\n", count);
```

Feedback: A counter starts at 0, because before the loop nothing has been counted yet.

(fib) L3 · Complete the sum: the operator

Complete the line so each value is added to the total. Type only the two-character operator.

```
int n, value;
int sum = 0;
printf("How many values? ");
scanf("%d", &n);
for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    sum [+=] value;
}
printf("Sum = %d\n", sum);
```

Feedback: `sum += value;` is the short form of `sum = sum + value;`. With a plain `=` the total would only hold the last value typed.

(fib) L3 · Complete the age check: the keyword

Complete the keyword so the program asks again as long as the age is bad.

```
int age;
printf("Enter age: ");
scanf("%d", &age);
[while] (age < 0 || age > 120) {
    printf("Invalid. Try again: ");
    scanf("%d", &age);
}
printf("Accepted\n");
```

Feedback: `while` repeats the block as long as the condition is true. With `if` the program would ask again only once, and a second bad age would be accepted.

(fib) L3 · Complete the password loop: the comparison

Complete the condition so the loop goes on while the code is **not** 1234. Type only the two-character operator.

```
int code;
```

```
printf("Password: ");
scanf("%d", &code);
while (code [!=] 1234) {
    printf("Wrong. Try again: ");
    scanf("%d", &code);
}
printf("Welcome\n");
```

Feedback: `!=` means "is not equal to". With `==` the loop would run only when the password is right, which is the opposite of what you want.

(fib) L3 · Complete the factorial: the operator

Complete the line so `fact` is multiplied by `i` on every repeat. Type only the two-character operator.

```
int fact = 1;
for (int i = 1; i <= n; i++) {
    fact [*=] i;
}
printf("%d! = %d\n", n, fact);
```

Feedback: `fact *= i;` is the short form of `fact = fact * i;`. For `n` = 4 the program prints `4! = 24`.

(fib) L3 · Complete the even numbers: the update

Complete the update so the loop prints only even numbers: 2, 4, 6, ... Type it with single spaces, exactly like `i += 1`.

```
for (int i = 2; i <= n; [i += 2|i+=2|i = i + 2|i=i+2]) {
    printf("%d ", i);
}
```

Feedback: The loop starts at 2 and adds 2 after every repeat. With `i++` it would print every number from 2 to `n`.

(fib) L3 · Trace the loop with a break

Trace the program by hand. What number does it print? Type only the number.

```
int total = 0;
for (int i = 1; i <= 10; i++) {
    if (i > 4) {
        break;
    }
    total += i;
}
printf("%d", total);
```

It prints [10].

Feedback: `total` grows by 1, 2, 3 and 4: that is 10. When `i` is 5, `i > 4` is true, `break;` ends the loop, and 5 is never added.